Final project - Full Stack Web Development

The final project assignment is to develop a web application and deploy it to the cloud. It should consist of a database, backend (server), and frontend (client).

Frontend should be done as a separate application using React library or another library / framework like Vue, Svelte, Angular, Next.js, etc. TypeScript is preferred and recommended but you can also use JavaScript.

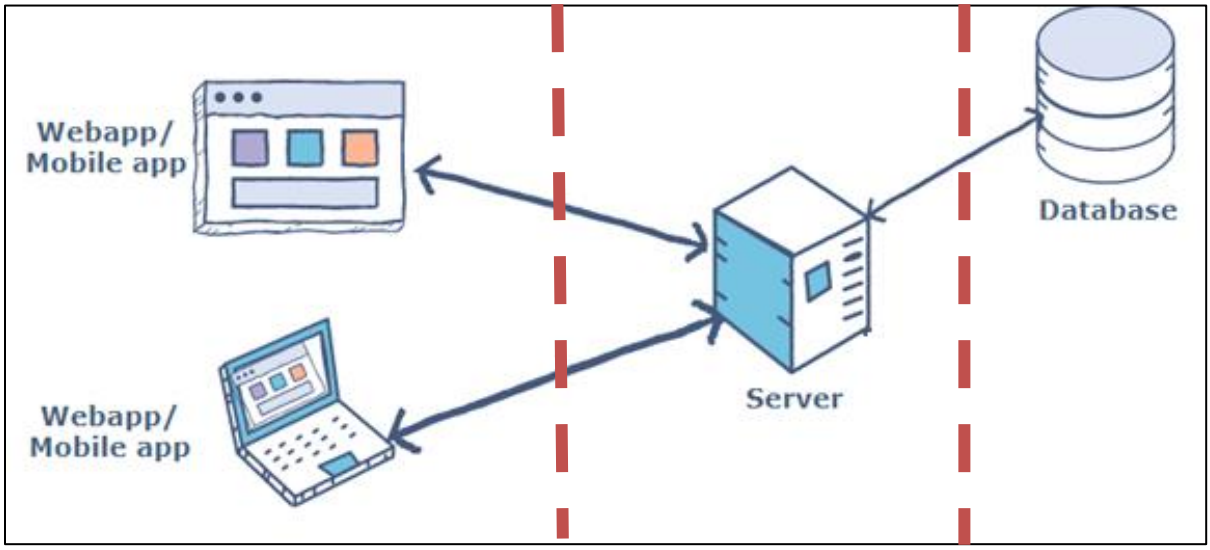
The backend should communicate with the frontend using REST APIs or GraphQL. You are free to choose any development stack – for example TypeScript (JavaScript) / Node.js, Java / Spring Boot, C# / ASP .NET, Python / Django, PHP / Laravel, …

You can use any database technology, just not SQLite because it is not a database server.

Here are some project examples just to get an idea (you can come up with your own idea):

- car rental system
- system for reviews of concerts and music releases
- banking system
- movie database like themoviedb.org
- video game database like rawg.io
- system for user reviews and recommendations of best restaurants etc – like yelp.com
- portfolio management system – for stock trading
- airline reservation system
- library management system
- web shop, job portal, art collection system

It should be complex enough to cover the curriculum of the whole course.
We are aiming for the following architecture - Database server, Web server, Web client:

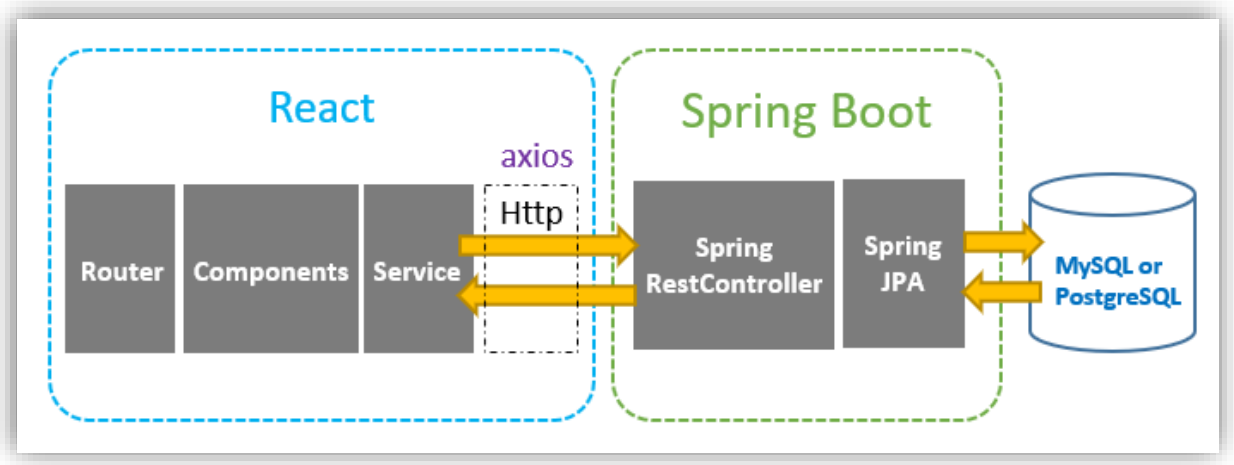


Our architecture offers separate hosting and scaling of the 3 parts – Database, backend application, client application.

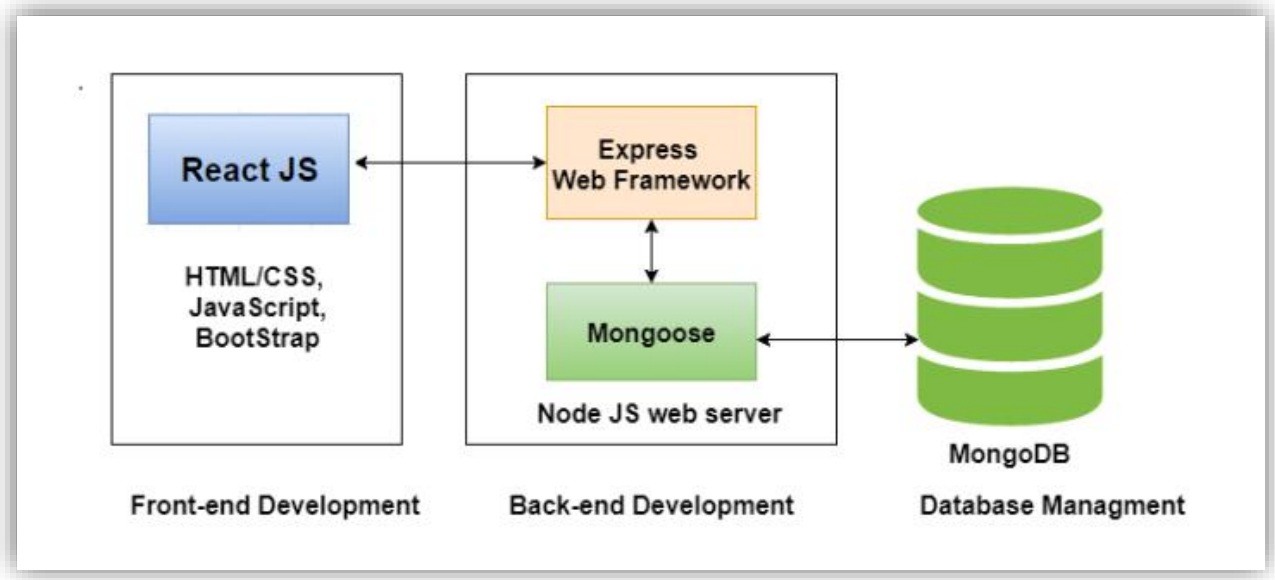
It is fine to work with monolithic system architecture, but you can also work with microservice architecture.

Examples of the system architecture and tech stack

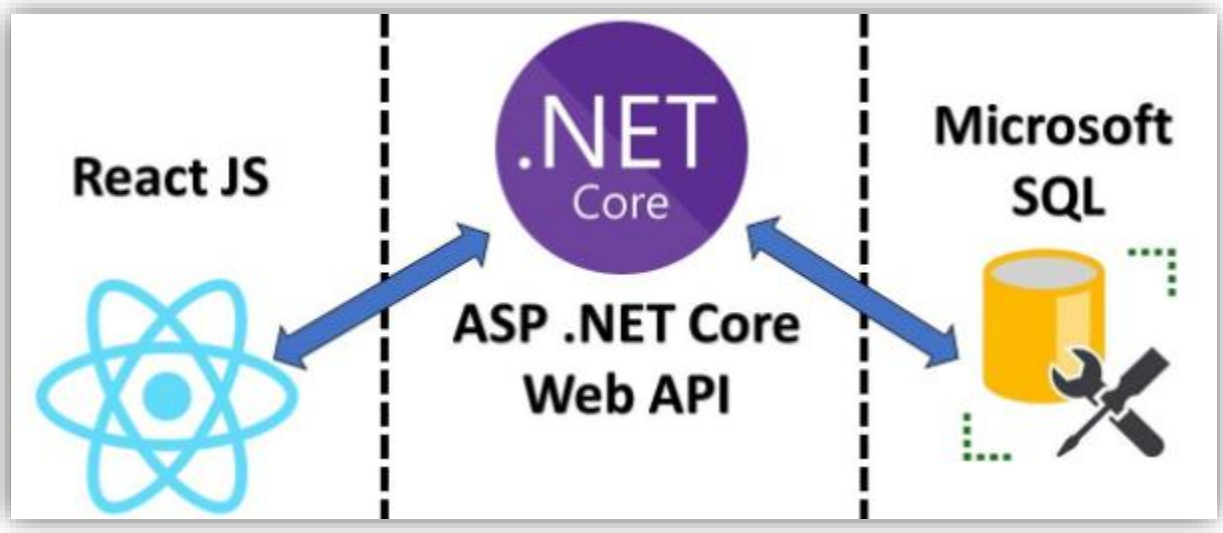
1. **React.js + Spring Boot + MySQL**



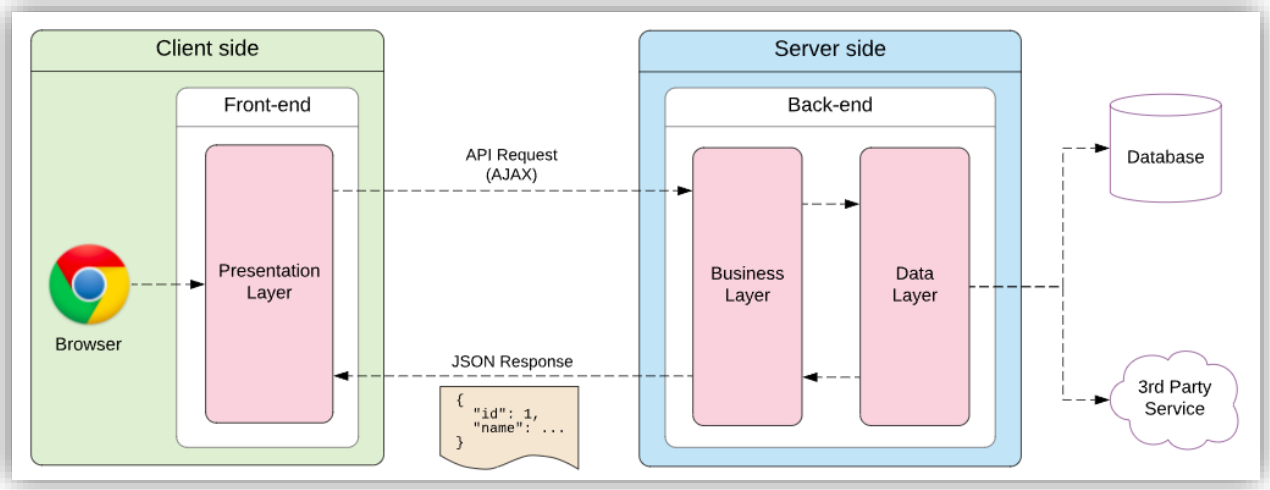
2. **React.js + Node.js + MongoDB (MERN stack):**



3. **React.js + ASP .NET + SQL Server:**



Single-page web application with REST APIs (you can also use GraphQL API)



List of things that should be implemented in the project

Optional parts are marked red.

- **Database:**
  - Any database technology – Relational database or NoSQL
  - Realistic test data (large enough data set)
  - Stored objects: Views, Stored procedures, Stored functions, Triggers, Events
  - Transactions for SQL statements that must be executed together
  - Scalability – using indexes to speed up crucial queries
  - Security: Definition of user accounts and privileges for the app and for admins
- **Backend:**
  - CRUD operations, error handling
  - REST API (Following the REST principles) or GraphQL
  - logging (for example sentry.io)
  - Testing – unit tests, integration tests
  - Transactions for SQL statements that must be executed together.
  - Security: SQL Injection protection, CORS – Cross-Origin Resource Sharing, HTTPS instead of HTTP, Authentication - JWT (HTTP-only cookies) or Session, Login and Registration APIs, password hashing
  - API documentation (OpenAPI Specification – Swagger, ReDoc)
- **Frontend:**
  - Proper architecture using components
  - SPA – Single-Page Application
  - Caching (cache time, stale time, initial data), request retries (for example using React Query)
  - Error handling
  - Routing – home page, error page, other pages, protected routes where needed
  - Logging service (for example sentry.io)
  - State management (props, context, global state) – proper solution
  - CSS Styles – using components, styled components, CSS files, etc.
  - Testing – unit tests, end to end tests (for example Cypress), …
  - Security measures against attacks: XSS – Cross-Site Scripting, CSRF – Cross-Site Request Forgery, Session hijacking; Login and Registration
  - Responsive design - Mobile-first approach
  - Sitemap
  - User Experience design
  - UI prototypes – for example using Figma
- Cloud deployment of the database, backend, and frontend:
  - Backend, frontend and database service should be deployed in the cloud.
  - CI/CD pipeline – for example: git → GitHub (on push, run the tests, if ok) → deploy to cloud (it can also contain creating docker images and pushing them to Dockerhub repository)
  - Using CDN (Content distribution network) for hosting static resources – images, videos, … (like AWS CloudFront)

Final delivery to WISEflow

- Short presentation of your project on 1 page that will also contain:
  - Links to GitHub repository / repositories with your source code.
  - Link to your deployed application.